

Selection sort — a formal proof of correctness

Written to be worked at the board. It leans on two results rather than reproving them: `idxOfMin`, proved in lecture 4/5, and `swap`, assumed.

The code

```
void selectionSort (vector<int>& C) {  
    for (int i = 0; i < C.size(); i++) {  
        int m = idxOfMin(C, i);  
        swap(C, m, i);  
    }  
}
```

Write n for `C.size()`, and C_0 for the contents of `C` on entry. Note n does not change: nothing in the loop resizes `C`.

What we are allowed to assume

Lemma 1 (`idxOfMin`). For $0 \leq a < n$, `idxOfMin(C, a)` returns an index ℓ with $a \leq \ell < n$ and

$$C[\ell] = \min_{a \leq j < n} C[j],$$

and does not modify `C`.

Lemma 2 (`swap`). `swap(C, p, q)` leaves `C` with $C'[p] = C[q]$, $C'[q] = C[p]$, and $C'[k] = C[k]$ for every $k \notin \{p, q\}$. In particular it is a transposition, so it permutes the contents of `C`.

What we are proving

Theorem. After `selectionSort(C)` returns, $\mathbf{C}$ is a permutation of C_0 and $C[0] \leq C[1] \leq \dots \leq C[n-1]$.

Both halves are needed. Sortedness alone is satisfied by a function that overwrites the array with zeroes.

The invariant

At the top of iteration i — meaning just before the guard is tested with the loop variable equal to i — for $0 \leq i \leq n$:

$P(i)$:

- (a) $\mathbf{C}$ is a permutation of C_0 ;
- (b) $C[0] \leq C[1] \leq \dots \leq C[i-1]$;
- (c) $\max_{0 \leq k < i} C[k] \leq \min_{i \leq j < n} C[j]$.

Clauses (b) and (c) are vacuous when $i = 0$; (c) is also vacuous when $i = n$.

Clause (c) is the one that does not appear in the theorem. It is carried purely to make the induction go through — see the remark at the end.

a) Initialization

Take $i = 0$.

- (a) No statement has executed, so $\mathbf{C}$ holds C_0 , and the identity is a permutation.
- (b) The range $C[0..-1]$ is empty, and the empty sequence is sorted.
- (c) There is no k with $0 \leq k < 0$, so the maximum is over an empty set and the claim is vacuous.

Hence $P(0)$. ■

b) Restoration

Assume $P(i)$ holds at the top of iteration i , and that the guard passed, so $i < n$. Let $\mathbf{C}$ denote the contents at the top of the iteration and C' the contents after the body runs. We show $P(i+1)$ holds of C' .

The body does two things:

$$m = \text{idxOfMin}(\mathbf{C}, i), \quad \text{swap}(\mathbf{C}, m, i).$$

By Lemma 1, $i \leq m < n$ and

$$C[m] = \min_{i \leq j < n} C[j], \quad (\star)$$

and $\mathbf{C}$ is unchanged by the call. By Lemma 2,

$$C'[i] = C[m], \quad C'[m] = C[i], \quad C'[k] = C[k] \text{ otherwise.}$$

(a) Permutation. idxOfMin does not modify $\mathbf{C}$, and swap is a transposition. A composition of permutations is a permutation, so by the inductive hypothesis C' is a permutation of C_0 .

(b) The prefix is sorted. We must show $C'[0] \leq \dots \leq C'[i]$.

Since $m \geq i$, the swap touches only indices $\geq i$, so

$$C'[k] = C[k] \quad \text{for all } k < i,$$

and by the inductive hypothesis (b) this prefix is sorted. It remains to check the join, that $C'[i-1] \leq C'[i]$, and only when $i \geq 1$.

By the inductive hypothesis (c),

$$C[i-1] \leq \max_{0 \leq k < i} C[k] \leq \min_{i \leq j < n} C[j] \stackrel{(\star)}{=} C[m] = C'[i],$$

and $C'[i-1] = C[i-1]$, so $C'[i-1] \leq C'[i]$ as required.

When $i = 0$ the range $C'[0..0]$ is a single element and is sorted trivially.

(c) The prefix stays below the suffix. We must show

$$\max_{0 \leq k < i+1} C'[k] \leq \min_{i+1 \leq j < n} C'[j].$$

By part (b) just proved, $C'[0..i]$ is sorted, so its maximum is its last entry:

$$\max_{0 \leq k \leq i} C'[k] = C'[i] = C[m].$$

For the right-hand side, compare the two suffixes as multisets. The swap moves $C[m]$ to position i and $C[i]$ to position m , and both i and m lie in $[i, n)$, so

$$\{C'[j] : i+1 \leq j < n\} = \{C[j] : i \leq j < n\} \setminus \{C[m]\}.$$

Every element of $\{C[j] : i \leq j < n\}$ is $\geq C[m]$ by $(\star)$, since $C[m]$ is the minimum of exactly that set. Removing one element cannot introduce anything smaller, so every remaining element is $\geq C[m] = C'[i]$, which is the claim.

Hence $P(i+1)$. ■

c) Termination

The loop variable starts at 0, increases by exactly 1 per iteration, and is tested against the fixed bound n . So after n iterations the guard fails and control leaves the loop with $i = n$.

$P(n)$ therefore holds, and reading it off:

- (a) C is a permutation of C_0 ;
- (b) $C[0] \leq C[1] \leq \dots \leq C[n-1]$ — the whole array is sorted;
- (c) vacuous.

Together, (a) and (b) are exactly the theorem. ■

Remark: why clause (c) is there

Drop (c) and the proof fails at one specific line. In part (b) we needed $C'[i-1] \leq C'[i]$, and the only thing that delivers it is knowing that everything already placed is no larger than anything still to come. Sortedness of the prefix alone says nothing about how the prefix compares to the rest, so the newly placed element could land below its predecessor.

So (c) is not part of what we set out to prove, and it is not used at termination — it retires there, vacuous. It exists solely so that the inductive step can be taken. That is strengthening the induction hypothesis: the statement you can prove is sometimes stronger than the one you want.

Remark: the last iteration

The loop runs to $i = n-1$, where `idxOfMin(C, n-1)` must return $n-1$ and the swap exchanges an element with itself. It is harmless, and the proof covers it without a special case, but the loop could equally run to $n-2$. Worth a sentence if someone asks — it is the same observation as the final pass of a bubble sort doing nothing.